

ECE 60146 HW 08

Andrea Goh

March 2025

1 mUnet

The output of mUnet is a mask corresponding to the input images. In the **PurdueShapes5** dataset, these masks consist of five channels, each representing one of the five object classes: rectangle, triangle, disk, oval, and star.

mUnet follows an encoder-decoder architecture. The encoder extracts object features by progressively reducing the spatial dimensions of the input image, allowing it to capture high-level representations. The decoder then reconstructs the spatial information by gradually upsampling the feature maps generated by the encoder. Skip connections transfer information between the encoder and decoder, helping to preserve pixel-level details, which improves decoder's performance.

2 Atrous Spatial Pyramid Pooling (ASPP)

2.1 Code Implementation

```
1 class mUnet(nn.Module):
2     def __init__(self, skip_connections=True, depth=16):
3         super(DLStudio.SemanticSegmentation.mUnet, self).__init__()
4         ...
5         ##### Added Initializations for ASPP #####
6         self.atrous1 = nn.Conv2d(128, 1, kernel_size=1, padding=0)
7         self.atrous2 = nn.Conv2d(128, 1, kernel_size=3, padding=2, dilation=2)
8         self.atrous3 = nn.Conv2d(128, 1, kernel_size=3, padding=4, dilation=4)
9         self.atrous4 = nn.Conv2d(128, 1, kernel_size=3, padding=6, dilation=6)
10        self.atrous_pool = nn.Conv2d(4, 1, kernel_size=1, padding=0)
11        self.reshape_conv = nn.Conv2d(1, 128, 1)
12        #####
13
14    def forward(self, x):
15        ## Going down to the bottom of the U:
16        x = nn.MaxPool2d(2,2)(nn.functional.relu(self.conv_in(x)))
17        ...
18        ##### Atrous Spatial Pyramid Pooling (ASPP) #####
19        x1 = self.atrous1(x)
20        x2 = self.atrous2(x)
21        x3 = self.atrous3(x)
22        x4 = self.atrous4(x)
23        x = torch.cat((x1, x2, x3, x4), dim=1)
24        x = self.atrous_pool(x)
25        x = self.reshape_conv(x)
26        #####
27        ...
28        ## Coming up from the bottom of U on the other side:
29        x = self.skip128usUP(x)
```

2.2 Running semantic segmentation.py with and without ASPP

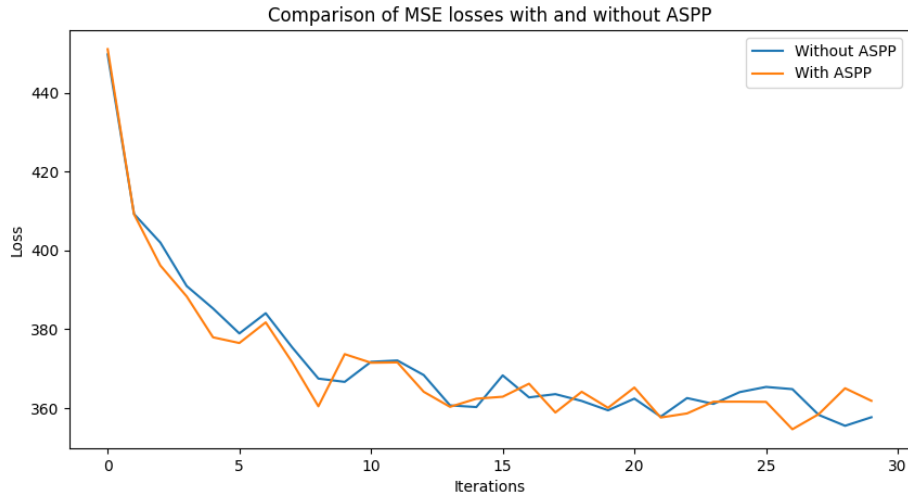


Figure 1: segmentation.py loss results with and without ASPP

When ASPP is added to the mUnet architecture, there is not much change in the loss curve. Both start and end at similar values and the downward trends are also similar.

3 Dice Loss

3.1 Implementation Code

```
1 def dice_loss(self, preds: torch.Tensor , ground_truth: torch. Tensor , epsilon=1e-6):
2
3     preds = preds.view(-1)
4     ground_truth = ground_truth.view(-1)
5
6     # Step1: Compute Dice Coefficient.
7     num = (preds*ground_truth).sum()
8     den = (preds*preds).sum() + (ground_truth*ground_truth).sum()
9
10    # Step2: dice_coeffecient = 2*numerator / (denominator + epsilon)
11    dice_coeffecient = 2*num / (den + epsilon)
12
13    # Step 3: Compute dice_loss = 1 - dice_coeffecient.
14    dice_loss = 1 - dice_coeffecient
15    return dice_loss
```

4 Results

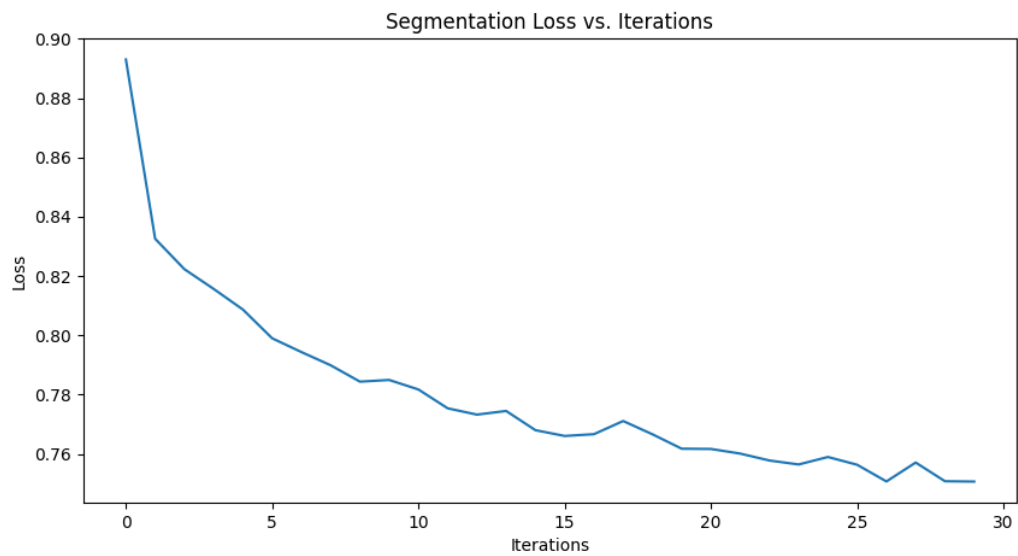


Figure 2: Loss Curve when only Dice Loss is used

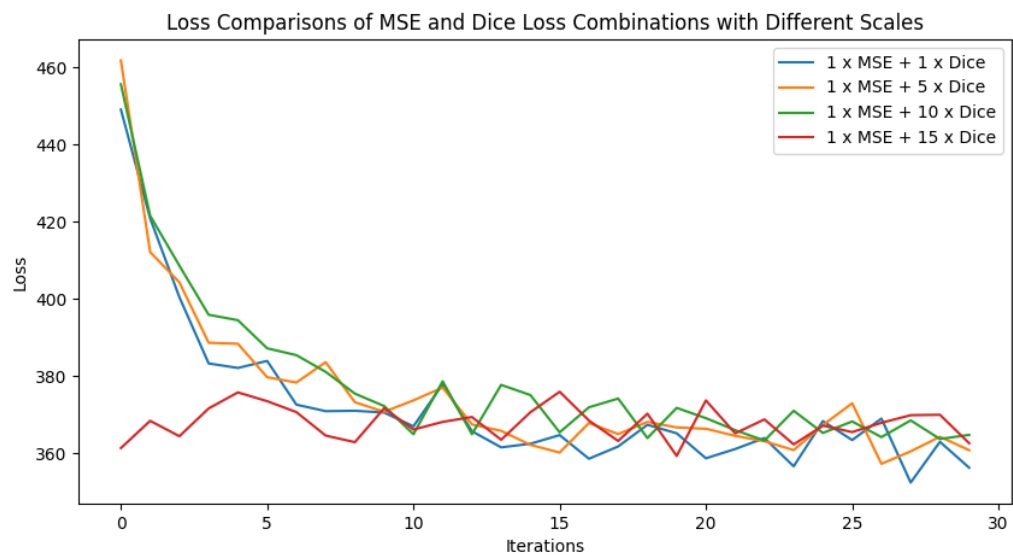
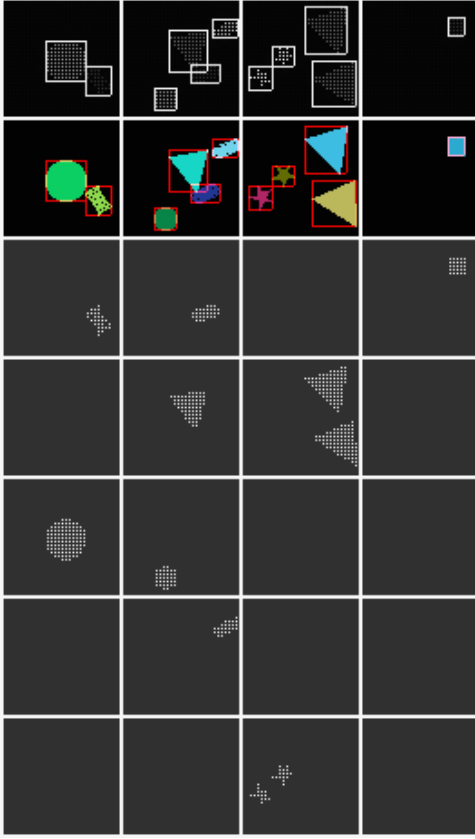
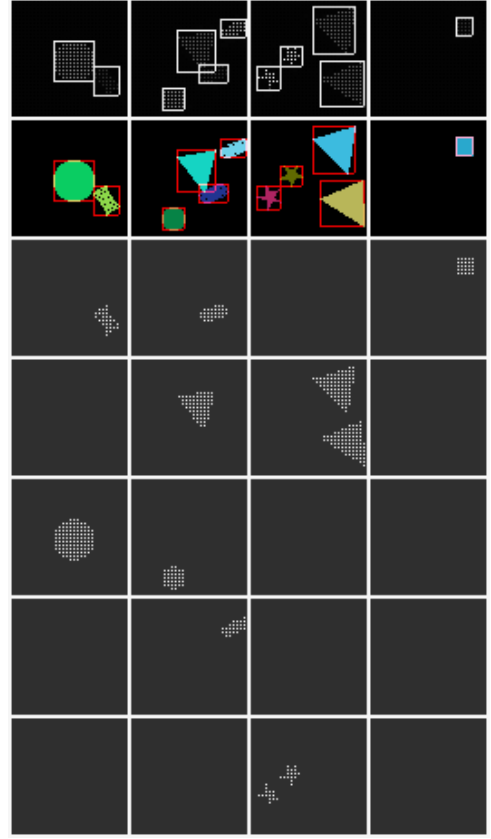


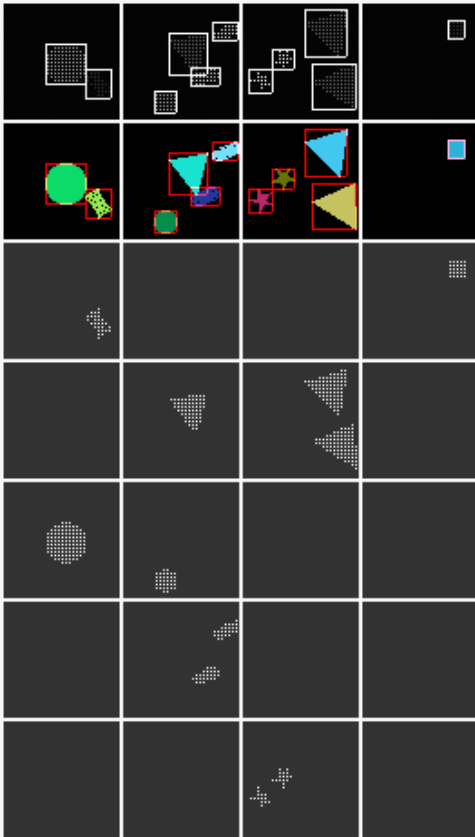
Figure 3: Loss Curves for Combinations of MSE and Dice Loss



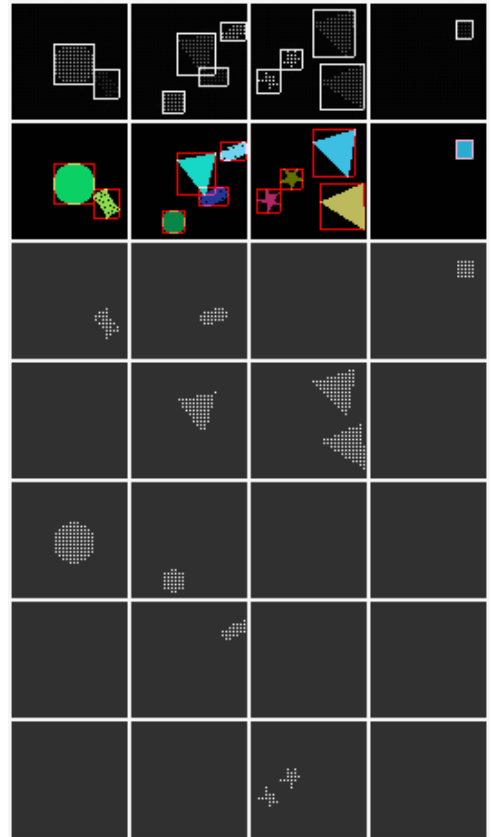
(a) MSE Loss only



(b) 1 x MSE + 1 x Dice



(c) 1 x MSE + 5 x Dice



(d) 1 x MSE + 10 x Dice

Figure 4: Segmentation results

4.1 Observations

When only the MSE loss is used, the object detection performs well. However, one of the two ovals are misclassified as a rectangle for the image in the second column. The same misclassification occurs when the MSE and dice losses are combined with the same scale. However, when the MSE and dice loss are combined at a scale of 1 and 5, the oval that was previously classified as a rectangle becomes correctly classified. When the scale on the dice loss is further increased to 10, one of the ovals again gets misclassified as a rectangle. Therefore, the best ratio between MSE and dice loss to use for the combined loss function is 1:5. From figure 3, we can see that when the dice factor is increased beyond 10, the loss does not converge. Therefore, the worst scale is weighing the dice with numbers larger than 10.

5 MSCOCO dataset

For the MSCOCO dataset, I was unable to obtain a stable loss curve.

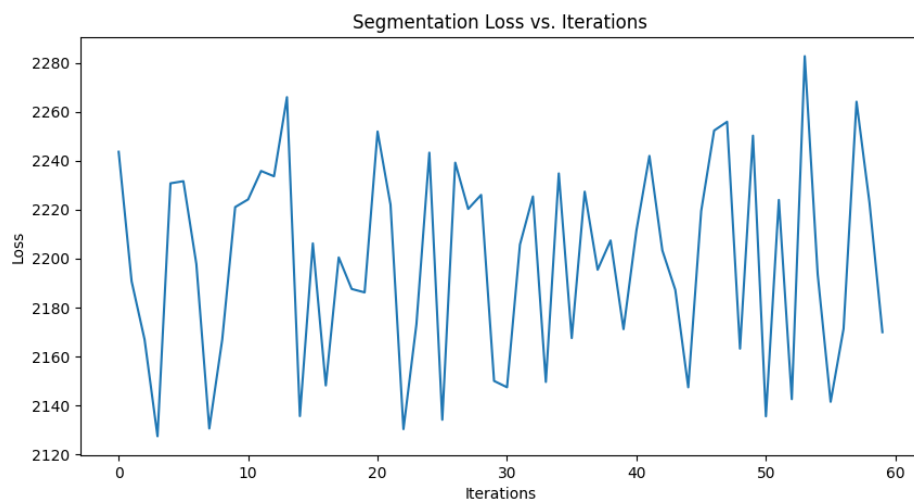


Figure 5: 1 x MSE + 1 x Dice

6 Source Code

6.1 Main Code

```
1 from DLStudio import *
2 import torch
3 import matplotlib.pyplot as plt
4 import torchvision.transforms as transforms
5 from PIL import Image
6 import torchvision.utils
7 import os
8 from pycocotools.coco import COCO
9 import cv2
10
11 class HW08(DLStudio.SemanticSegmentation):
12     from DLStudio import *
13     import torch
14     import matplotlib.pyplot as plt
15     import torchvision.transforms as transforms
16     from PIL import Image
17     import torchvision.utils
18     import os
19     from pycocotools.coco import COCO
20
21 class HW08(DLStudio.SemanticSegmentation):
22
```

```

23     def dice_loss(self, preds: torch.Tensor , ground_truth: torch. Tensor , epsilon=1e
24         -6):
25
26         preds = preds.view(-1)
27         ground_truth = ground_truth.view(-1)
28
29         # Step1: Compute Dice Coefficient.
30         num = (preds*ground_truth).sum()
31         den = (preds*preds).sum() + (ground_truth*ground_truth).sum()
32
33         # Step2: dice_coefecient = 2*numerator / (denominator + epsilon)
34         dice_coefecient = 2*num / (den + epsilon)
35
36         # Step 3: Compute dice_loss = 1 - dice_coefecient.
37         dice_loss = 1 - dice_coefecient
38         return dice_loss
39
40     def run_code_for_training_for_semantic_segmentation(self, net, scale, netname):
41         self.netname = netname
42         filename_for_out1 = f"performance_numbers_{netname}.txt"
43         FILE1 = open(filename_for_out1, 'w')
44         net = copy.deepcopy(net)
45         net = net.to(self.dl_studio.device)
46         criterion1 = nn.MSELoss()
47         optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
48         # optimizer = optim.SGD(net.parameters(),
49         #                         lr=self.dl_studio.learning_rate, momentum=self.dl_studio.
50         #                         momentum)
51         segmentation_loss_tally = []
52         start_time = time.perf_counter()
53         for epoch in range(self.dl_studio.epochs):
54             print("")
55             running_loss_segmentation = 0.0
56             for i, data in enumerate(self.train_dataloader):
57                 im_tensor,mask_tensor,bbox_tensor =data['image'],data['mask_tensor'],
58                 data['bbox_tensor']
59                 im_tensor = im_tensor.to(self.dl_studio.device)
60                 mask_tensor = mask_tensor.type(torch.FloatTensor)
61                 mask_tensor = mask_tensor.to(self.dl_studio.device)
62                 bbox_tensor = bbox_tensor.to(self.dl_studio.device)
63                 optimizer.zero_grad()
64                 output = net(im_tensor)
65                 mse_loss = criterion1(output, mask_tensor)
66                 dice_loss = self.dice_loss(preds=output, ground_truth=mask_tensor)
67                 segmentation_loss = scale[0]*mse_loss + scale[1] * dice_loss
68                 segmentation_loss.backward()
69                 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
70                 optimizer.step()
71                 running_loss_segmentation += segmentation_loss.item()
72                 if i%500==499:
73                     current_time = time.perf_counter()
74                     elapsed_time = current_time - start_time
75                     avg_loss_segmentation = running_loss_segmentation / float(500)
76                     segmentation_loss_tally.append(avg_loss_segmentation)
77                     print("[epoch=%d/%d,iter=%4d,elapsed_time=%3d,secs]MSE_loss:
78                     %.3f" % (epoch+1, self.dl_studio.epochs, i+1, elapsed_time,
79                     avg_loss_segmentation))
80                     FILE1.write("%.3f\n" % avg_loss_segmentation)
81                     FILE1.flush()
82                     running_loss_segmentation = 0.0
83             print("\nFinished Training\n")
84             torch.save(net.state_dict(), f"saved_model_{self.netname}")
85             plt.figure(figsize=(10,5))
86             plt.plot(segmentation_loss_tally, label=self.netname)
87             plt.xlabel("Iterations")
88             plt.ylabel("Loss")
89             plt.title("Segmentation Loss vs. Iterations")
90             plt.legend
91             plt.savefig(f"{netname}.png")
92
93     def run_code_for_testing_semantic_segmentation(self, net, netname):
94         net.load_state_dict(torch.load(f"saved_model_{netname}"))
95         batch_size = self.dl_studio.batch_size

```

```

91     image_size = self.dl_studio.image_size
92     max_num_objects = self.max_num_objects
93     save_count = 0
94     with torch.no_grad():
95         for i, data in enumerate(self.test_dataloader):
96             im_tensor, mask_tensor, bbox_tensor = data['image'], data['mask_tensor'],
97                 data['bbox_tensor']
98             if i % 50 == 0:
99                 print("\n\n\nShowing output for test batch %d:" % (i+1))
100                 outputs = net(im_tensor)
101                 ## In the statement below: 1st arg for batch items, 2nd for channels
102                 , 3rd and 4th for image size
103                 output_bw_tensor = torch.zeros(batch_size, 1, image_size[0], image_size
104                     [1], dtype=float)
105                 for image_idx in range(batch_size):
106                     for layer_idx in range(max_num_objects):
107                         for m in range(image_size[0]):
108                             for n in range(image_size[1]):
109                                 output_bw_tensor[image_idx, 0, m, n] = torch.max(
110                                     outputs[image_idx, :, m, n] )
111                 display_tensor = torch.zeros(7 * batch_size, 3, image_size[0],
112                     image_size[1], dtype=float)
113                 for idx in range(batch_size):
114                     for bbox_idx in range(max_num_objects):
115                         bb_tensor = bbox_tensor[idx, bbox_idx]
116                         for k in range(max_num_objects):
117                             i1 = int(bb_tensor[k][1])
118                             i2 = int(bb_tensor[k][3])
119                             j1 = int(bb_tensor[k][0])
120                             j2 = int(bb_tensor[k][2])
121                             if i1 > 255: j2 == 255
122                             if i2 > 255: j2 == 255
123                             if j1 > 255: j2 == 255
124                             if j2 > 255: j2 == 255
125                             output_bw_tensor[idx, 0, i1:i2, j1] = 255
126                             output_bw_tensor[idx, 0, i1:i2, j2] = 255
127                             output_bw_tensor[idx, 0, i1, j1:j2] = 255
128                             output_bw_tensor[idx, 0, i2, j1:j2] = 255
129                             im_tensor[idx, 0, i1:i2, j1] = 255
130                             im_tensor[idx, 0, i1:i2, j2] = 255
131                             im_tensor[idx, 0, i1, j1:j2] = 255
132                             im_tensor[idx, 0, i2, j1:j2] = 255
133                 display_tensor[:batch_size, :, :, :] = output_bw_tensor
134                 display_tensor[batch_size:2*batch_size, :, :, :] = im_tensor
135
136                 for batch_im_idx in range(batch_size):
137                     for mask_layer_idx in range(max_num_objects):
138                         for i in range(image_size[0]):
139                             for j in range(image_size[1]):
140                                 if mask_layer_idx == 0:
141                                     if 25 < outputs[batch_im_idx, mask_layer_idx, i, j]
142                                         < 85:
143                                         outputs[batch_im_idx, mask_layer_idx, i, j] =
144                                             255
145                                 else:
146                                     outputs[batch_im_idx, mask_layer_idx, i, j] =
147                                         50
148                                 elif mask_layer_idx == 1:
149                                     if 65 < outputs[batch_im_idx, mask_layer_idx, i, j]
150                                         < 135:
151                                     outputs[batch_im_idx, mask_layer_idx, i, j] =
152                                         255
153                                 else:
154                                     outputs[batch_im_idx, mask_layer_idx, i, j] =
155                                         50
156                                 elif mask_layer_idx == 2:
157                                     if 115 < outputs[batch_im_idx, mask_layer_idx, i, j]
158                                         < 185:
159                                     outputs[batch_im_idx, mask_layer_idx, i, j] =
160                                         255
161                                 else:
162                                     outputs[batch_im_idx, mask_layer_idx, i, j] =
163                                         50

```

```

150         elif mask_layer_idx == 3:
151             if 165 < outputs[batch_im_idx,mask_layer_idx,i,j]
152                 < 230:
153                 outputs[batch_im_idx,mask_layer_idx,i,j] =
154                     255
155             else:
156                 outputs[batch_im_idx,mask_layer_idx,i,j] =
157                     50
158             elif mask_layer_idx == 4:
159                 if outputs[batch_im_idx,mask_layer_idx,i,j] >
160                     210:
161                 outputs[batch_im_idx,mask_layer_idx,i,j] =
162                     255
163             else:
164                 outputs[batch_im_idx,mask_layer_idx,i,j] =
165                     50
166
167         display_tensor[2*batch_size+batch_size*mask_layer_idx+
168             batch_im_idx,:,:,:]= outputs[batch_im_idx,mask_layer_idx
169             ,,:,:]
170
171         grid_image = torchvision.utils.make_grid(display_tensor, nrow=
172             batch_size, normalize=True, padding=2, pad_value=10)
173         transform = transforms.ToPILImage()
174         pil_image = transform(grid_image.cpu())
175         save_dir = f"Image_Results_{netname}"
176         os.makedirs(save_dir, exist_ok=True)
177         image_path = os.path.join(save_dir, f"segmentation_results_batch_{
178             save_count}.png")
179         save_count+=1
180         pil_image.save(image_path)
181
182     class CustomCOCODataset(torch.utils.data.Dataset):
183         def __init__(self, dl_studio, segmenter, train_or_test, dataset_file):
184             super(HW08.CustomCOCODataset, self).__init__()
185             max_num_objects = segmenter.max_num_objects
186             if train_or_test == 'train' and dataset_file == "CustomCOCO_5226_train.gz":
187                 if os.path.exists("torch_saved_CustomCOCO_5226_dataset.pt") and \
188                     os.path.exists("torch_saved_CustomCOCO_label_map.pt"):
189                     print("\nLoading training data from torch saved file")
190                     self.dataset = torch.load("torch_saved_CustomCOCO_5226_dataset.pt",
191                         weights_only=False)
192                     self.label_map = torch.load("torch_saved_CustomCOCO_label_map.pt")
193                     self.num_shapes = len(self.label_map)
194                     self.image_size = dl_studio.image_size
195             else:
196                 print("""\n\n\nLooks like this is the first time you will be loading
197                     in\n\n\n
198                     ""the dataset for this script. First time loading could
199                     take\n\n\n
200                     ""a few minutes. Any subsequent attempts will only take\n
201                     ""\n
202                     ""a few seconds.\n\n\n""")
203                 root_dir = dl_studio.dataroot
204                 f = gzip.open(root_dir + dataset_file, 'rb')
205                 dataset = f.read()
206                 self.dataset, self.label_map = pickle.loads(dataset, encoding='
207                     latin1')
208                 torch.save(self.dataset, "torch_saved_CustomCOCO_5226_dataset.pt")
209                 torch.save(self.label_map, "torch_saved_CustomCOCO_label_map.pt")
210                 # reverse the key-value pairs in the label dictionary:
211                 self.class_labels = dict(map(reversed, self.label_map.items()))
212                 self.num_shapes = len(self.class_labels)
213                 self.image_size = dl_studio.image_size
214             else:
215                 root_dir = dl_studio.dataroot
216                 f = gzip.open(root_dir + dataset_file, 'rb')
217                 dataset = f.read()
218                 if sys.version_info[0] == 3:
219                     self.dataset, self.label_map = pickle.loads(dataset, encoding='
220                         latin1')
221                 else:
222                     self.dataset, self.label_map = pickle.loads(dataset)
223                 # reverse the key-value pairs in the label dictionary:

```

```

207         self.class_labels = dict(map(reversed, self.label_map.items()))
208         self.num_shapes = len(self.class_labels)
209         self.image_size = dl_studio.image_size
210
211     def __len__(self):
212         return len(self.dataset)
213
214     def __getitem__(self, idx):
215         image_size = self.image_size
216         r = np.array( self.dataset[idx][0] )
217         g = np.array( self.dataset[idx][1] )
218         b = np.array( self.dataset[idx][2] )
219         R,G,B = r.reshape(image_size[0],image_size[1]), g.reshape(image_size[0],
220             image_size[1]), b.reshape(image_size[0],image_size[1])
221         im_tensor = torch.zeros(3,image_size[0],image_size[1], dtype=torch.float)
222         im_tensor[0,:,:] = torch.from_numpy(R)
223         im_tensor[1,:,:] = torch.from_numpy(G)
224         im_tensor[2,:,:] = torch.from_numpy(B)
225         mask_array = np.array(self.dataset[idx][3])
226         max_num_objects = len( mask_array[0] )
227         mask_tensor = torch.from_numpy(mask_array)
228         mask_val_to_bbox_map = self.dataset[idx][4] # bbox
229         max_bboxes_per_entry_in_map = max([ len(mask_val_to_bbox_map[key]) for key
230             in mask_val_to_bbox_map ])
231         ## The first arg 5 is for the number of bboxes we are going to need. If all
232         ## the
233         ## shapes are exactly the same, you are going to need five different bbox'
234         ## es.
235         ## The second arg is the index reserved for each shape in a single bbox
236         bbox_tensor = torch.zeros(max_num_objects,self.num_shapes,4, dtype=torch.
237             float)
238         class_to_index = {50: 0, 100: 1, 150: 2}
239         for bbox_idx in range(max_bboxes_per_entry_in_map): # range(3)
240             for key in mask_val_to_bbox_map: # 50, 100, 150
241                 if len(mask_val_to_bbox_map[key]) == 1: # number of bbox
242                     if bbox_idx == 0: #
243                         # print(key)
244                         # print(torch.from_numpy(np.array(mask_val_to_bbox_map[key][
245                             bbox_idx])))
246                         bbox_tensor[bbox_idx,class_to_index[key],:] = torch.
247                             from_numpy(np.array(mask_val_to_bbox_map[key][bbox_idx]
248                                 ))
249                     elif len(mask_val_to_bbox_map[key]) > 1 and bbox_idx < len(
250                         mask_val_to_bbox_map[key]):
251                         bbox_tensor[bbox_idx,class_to_index[key],:] = torch.from_numpy(
252                             np.array(mask_val_to_bbox_map[key][bbox_idx]))
253             sample = {'image' : im_tensor,
254                 'mask_tensor' : mask_tensor,
255                 'bbox_tensor' : bbox_tensor }
256         return sample
257
258     def load_CustomCOCO_dataset(self, dataservert_train, dataservert_test ):
259         self.train_dataloader = torch.utils.data.DataLoader(dataservert_train,
260             batch_size=self.dl_studio.batch_size,shuffle=True, num_workers=4)
261         self.test_dataloader = torch.utils.data.DataLoader(dataservert_test,
262             batch_size=self.dl_studio.batch_size,shuffle=False,
263             num_workers=4)
264
265     def compare_losses(losses, labels, title):
266         plt.figure(figsize=(10,5))
267         for i, loss_tally in enumerate(losses):
268             plt.plot(loss_tally, label=labels[i])
269         plt.xlabel("Iterations")
270         plt.ylabel("Loss")
271         plt.title(title)
272         plt.legend()
273         plt.savefig(f"{title}.png")
274         plt.cla()
275
276     def get_loss_tally_in_list(filename):
277         with open(filename, "r") as file:
278             loss_tally = [float(line.strip()) for line in file]
279         return loss_tally

```

```

269
270
271 ##### PurdueShapes5 Dataset #####
272 dls = DLStudio(
273     dataroot = "../data/",
274     image_size = [64,64],
275     path_saved_model = "../saved_model",
276     momentum = 0.9,
277     learning_rate = 1e-4,
278     epochs = 6,
279     batch_size = 4,
280     classes = ('rectangle', 'triangle', 'disk', 'oval', 'star'),
281     use_gpu = True,
282 )
283
284 segmenter = DLStudio.SemanticSegmentation(
285     dl_studio = dls,
286     max_num_objects = 5,
287 )
288
289 dataserer_train = DLStudio.SemanticSegmentation.PurdueShapes5MultiObjectDataset(
290     train_or_test = 'train',
291     dl_studio = dls,
292     segmenter = segmenter,
293     dataset_file = "PurdueShapes5MultiObject-10000-train.gz",
294 )
295
296 dataserer_test = DLStudio.SemanticSegmentation.PurdueShapes5MultiObjectDataset(
297     train_or_test = 'test',
298     dl_studio = dls,
299     segmenter = segmenter,
300     dataset_file = "PurdueShapes5MultiObject-1000-test.gz"
301 )
302
303 hw08_segmenter = HW08(dl_studio=dls, max_num_objects=5)
304
305 hw08_segmenter.load_PurdueShapes5MultiObject_dataset(dataserer_train, dataserer_test)
306
307 model = hw08_segmenter.mUNet(skip_connections=True, depth=16)
308
309 ## Compare loss functions with and without ASPP for MSE
310 losses = []
311 labels = []
312 # Comment out ASPP code block and run the next four lines
313 hw08_segmenter.run_code_for_training_for_semantic_segmentation(model, [1,0], 'MSE')
314 losses.append(get_loss_tally_in_list('performance_numbers_MSE.txt'))
315 labels.append('Without ASPP')
316 hw08_segmenter.run_code_for_testing_semantic_segmentation(model, 'MSE')
317 # Uncomment ASPP code block and run the remaining lines
318 hw08_segmenter.run_code_for_training_for_semantic_segmentation(model, [1,0], 'MSE_atrous
319 ')
320 losses.append(get_loss_tally_in_list('performance_numbers_MSE_atrous.txt'))
321 labels.append('With ASPP')
322 compare_losses(losses, labels, 'Comparison of MSE losses with and without ASPP')
323 hw08_segmenter.run_code_for_testing_semantic_segmentation(model, 'MSE_atrous')
324
325 # For the rest of the assignment, the ASPP version will be in use
326 scales = [[1,1], [1,5], [1,10], [1,15], [0,1]]
327 netnames = ['1_1', '1_5', '1_10', '1_15', 'dice']
328
329 losses = []
330 labels = []
331
332 for i, scale in enumerate(scales):
333     netname = netnames[i]
334     hw08_segmenter.run_code_for_training_for_semantic_segmentation(model, scale, netname
335 )
336 losses.append(get_loss_tally_in_list(f'performance_numbers_{netname}.txt'))
337 labels.append(f'{scale[0]}xMSE_{scale[1]}xDice')
338 hw08_segmenter.run_code_for_testing_semantic_segmentation(model, netname)
339 compare_losses(losses, labels, 'Loss Comparisons of MSE and Dice Loss Combinations with
340 Different Scales')

```

```

339
340 ##### COCO Dataset #####
341 dls = DLStudio(
342     dataroot = "./data/",
343     image_size = [256,256],
344     path_saved_model = "./saved_model",
345     momentum = 0.9,
346     learning_rate = 1e-4,
347     epochs = 6,
348     batch_size = 4,
349     classes = ('pizza', 'cat', 'bus'),
350     use_gpu = True,
351 )
352
353 segmenter = DLStudio.SemanticSegmentation(
354     dl_studio = dls,
355     max_num_objects = 3,
356 )
357
358 dataserver_train = HW08.CustomCOCODataset(
359     train_or_test = 'train',
360     dl_studio = dls,
361     segmenter = segmenter,
362     dataset_file = "CustomCOCO_5226_train.gz",
363 )
364
365 dataserver_test = HW08.CustomCOCODataset(
366     train_or_test = 'test',
367     dl_studio = dls,
368     segmenter = segmenter,
369     dataset_file = "CustomCOCO_2639_val.gz"
370 )
371
372 hw08_segmenter = HW08(dl_studio=dls, max_num_objects=3)
373
374 hw08_segmenter.load_CustomCOCO_dataset(dataserver_train, dataserver_test)
375
376 model = hw08_segmenter.mUNet(skip_connections=True, depth=16)
377
378 scales = [[1,1], [1,5], [1,10], [1,15]]
379 netnames = ['MSE', '1_1', '1_5', '1_10', '1_15']
380
381 losses = []
382 labels = []
383
384 for i, scale in enumerate(scales):
385     netname = netnames[i]
386     hw08_segmenter.run_code_for_training_for_semantic_segmentation(model, scale, netname)
387     losses.append(get_loss_tally_in_list(f'performance_numbers_{netname}.txt'))
388     labels.append(f'{scales[0]}xMSE+{scales[1]}xDice')
389     # hw08_segmenter.run_code_for_testing_semantic_segmentation(model, netname)
390
391 compare_losses(losses, labels, 'Loss Comparisons of MSE and Dice Loss Combinations with Different Scales')
392
393 # for MSE and dice alone
394 hw08_segmenter.run_code_for_training_for_semantic_segmentation(model, [1,0], 'MSE')
395 hw08_segmenter.run_code_for_testing_semantic_segmentation(model, [1,0])
396
397 hw08_segmenter.run_code_for_training_for_semantic_segmentation(model, [0,1], 'dice')
398 hw08_segmenter.run_code_for_testing_semantic_segmentation(model, [0,1])

```

6.2 Data Generation Code

```

1 from pycocotools.coco import COCO
2 import numpy as np
3 import cv2
4 import os
5 import gzip
6 import pickle

```

```

7
8 def generate_dataset(type):
9     input_json = f'annotations/instances_{type}2014.json'
10    image_directory = f'{type}2014'
11    class_list = ['pizza', 'cat', 'bus']
12
13    # Mapping from COCO label to class indices
14    coco_labels_inverse = {}
15    coco = COCO(input_json)
16    catIds = coco.getCatIds(catNms=class_list)
17    categories = coco.loadCats(catIds)
18    categories.sort(key=lambda x: x['id'])
19    coco_labels_inverse = {c['id']: idx for idx, c in enumerate(categories)}
20
21    for idx, in_class in enumerate(class_list):
22        for c in categories:
23            if c['name'] == in_class:
24                coco_labels_inverse[c['id']] = idx
25
26    imgIds = set() # use set to avoid duplication of images
27    # Retrieve Image list
28    for catId in catIds:
29        imgIds.update(coco.getImgIds(catIds=catId))
30
31    filtered_img_ids = [] # Stores valid image IDs
32
33    for img_id in imgIds:
34        ann_ids = coco.getAnnIds(imgIds=img_id, iscrowd=False)
35        anns = coco.loadAnns(ann_ids)
36
37        # Get unique categories in the image
38        obj_counts = {}
39        for ann in anns:
40            obj_category = coco.loadCats(ann['category_id'])[0]['name']
41            obj_counts[obj_category] = obj_counts.get(obj_category, 0) + 1
42
43        # Check if the image contains more than one type of object
44        if len(obj_counts) > 1:
45            # Check for large foreground objects
46            has_large_foreground = False
47            for ann in anns:
48                area = ann['area'] # COCO format: [x_min, y_min, width, height]
49                if area >= 40000: # Check if area is greater than 40,000 pixels
50                    has_large_foreground = True
51                    break # No need to check further, found a valid object
52
53            if has_large_foreground:
54                filtered_img_ids.append(img_id)
55
56    dataset = {}
57
58    for i, img_id in enumerate(filtered_img_ids):
59
60        img_info = coco.loadImgs(img_id)[0]
61        img_path = os.path.join(image_directory, img_info["file_name"])
62        img = cv2.imread(img_path)
63        H, W = img.shape[:2]
64        img_resized = cv2.resize(img, (256, 256))
65        R = img_resized[:, :, 0].flatten().tolist()
66        G = img_resized[:, :, 1].flatten().tolist()
67        B = img_resized[:, :, 2].flatten().tolist()
68
69        ann_ids = coco.getAnnIds(imgIds=img_id, iscrowd=False)
70        anns = coco.loadAnns(ann_ids)
71        bbox_dict = {50: [], 100: [], 150: []}
72        mask = np.zeros((3, 256, 256), dtype=np.uint8)
73        for ann in anns:
74            if ann['category_id'] not in catIds:
75                continue
76            shape_index = coco_labels_inverse[ann['category_id']]
77            x, y, w, h = ann['bbox']
78            bbox = [x* 256/W, y* 256/H, (x+w)* 256/W, (y+h)* 256/H]
79            bbox_dict[(shape_index+1)*50].append(bbox)

```

```

80         temp_mask = coco.annToMask(ann)
81         temp_mask_resize = cv2.resize(temp_mask, (256, 256), interpolation=cv2.
            INTER_NEAREST)
82         mask[shape_index] = np.maximum(mask[shape_index], temp_mask_resize * (50 * (
            shape_index + 1))) # Assign correct label
83
84         dataset[i] = {
85             0: R,
86             1: G,
87             2: B,
88             3: mask,
89             4: bbox_dict
90         }
91         label_map = {
92             categories[0]['name']: 50,
93             categories[1]['name']: 100,
94             categories[2]['name']: 150
95         }
96
97         with gzip.open(f'data/CustomCOCO_{len(filtered_img_ids)}_{type}.gz', "wb") as f:
98             pickle.dump((dataset, label_map), f)
99
100 generate_dataset('train')
101 generate_dataset('val')

```
